agriculture

MDPI

Article

A Hybrid Path Planning Algorithm for Orchard Robots Based on an Improved D* Lite Algorithm

Quanjie Jiang, Yue Shen, Hui Liu *, Zohaib Khan, Hao Sun and Yuxuan Huang

School of Electrical and Information Engineering, Jiangsu University, Zhenjiang 212013, China; 2212307079@stmail.ujs.edu.cn (Q.J.); shen@ujs.edu.cn (Y.S.); 5103220306@stmail.ujs.edu.cn (Z.K.); 2222207058@stmail.ujs.edu.cn (H.S.); 2212307050@stmail.ujs.edu.cn (Y.H.)
* Correspondence: amity@ujs.edu.cn

Abstract

Due to the complex spatial structure, dense tree distribution, and narrow passages in orchard environments, traditional path planning algorithms often struggle with large path deviations, frequent turning, and reduced navigational safety. In order to overcome these challenges, this paper proposes a hybrid path planning algorithm based on improved D* Lite for narrow forest orchard environments. The proposed approach enhances path feasibility and improves the robustness of the navigation system. The algorithm begins by constructing a 2D grid map reflecting the orchard layout and inflates the tree regions to create safety buffers for reliable path planning. For global path planning, an enhanced D* Lite algorithm is used with a cost function that jointly considers centerline proximity, turning angle smoothness, and directional consistency. This guides the path to remain close to the orchard row centerline, improving structural adaptability and path rationality. Narrow passages along the initial path are detected, and local replanning is performed using a Hybrid A* algorithm that accounts for the kinematic constraints of a differential tracked robot. This generates curvature-continuous and directionally stable segments that replace the original narrow-path portions. Finally, a gradient descent method is applied to smooth the overall path, improving trajectory continuity and execution stability. Field experiments in representative orchard environments demonstrate that the proposed hybrid algorithm significantly outperforms traditional D* Lite and KD* Lite-B methods in terms of path accuracy and navigational safety. The average deviation from the centerline is only 0.06 m, representing reductions of 75.55% and 38.27% compared to traditional D* Lite and KD* Lite-B, respectively, thereby enabling high-precision centerline tracking. Moreover, the number of hazardous nodes, defined as path points near obstacles, was reduced to five, marking decreases of 92.86% and 68.75%, respectively, and substantially enhancing navigation safety. These results confirm the method's strong applicability in complex, constrained orchard environments and its potential as a foundation for efficient, safe, and fully autonomous agricultural robot operation.

Keywords: orchard robot; path planning; D* lite; Hybrid A*; trajectory optimization

Academic Editor: Jiehao Li

Received: 7 July 2025
Revised: 1 August 2025
Accepted: 5 August 2025
Published: 6 August 2025

Citation: Jiang, Q.; Shen, Y.; Liu, H.; Khan, Z.; Sun, H.; Huang, Y. A Hybrid Path Planning Algorithm for Orchard Robots Based on an Improved D* Lite Algorithm. *Agriculture* **2025**, *15*, 1698. https://doi.org/10.3390/agriculture15151698

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

As agricultural production shifts toward high-efficiency, data-driven systems, the transformation of orchards into intelligent and automated environments has become a pressing objective [1–4]. Unlike open farmlands, orchards exhibit uniquely constrained

and irregular terrains, characterized by narrow passages, densely planted trees, and unstructured undergrowth [5–7]. These features challenge traditional mechanization and call for advanced robotic mobility. In such scenarios, it becomes mission-critical for robots not only to plan feasible paths but also to execute them reliably. Any deviation can lead to operational failure, crop damage, or inefficient task coverage—especially during precision spraying, fruit monitoring, or pruning operations [8,9]. The growing complexity of orchard environments has accelerated the integration of autonomous robotic systems into agricultural workflows, where robust path planning plays a crucial role in intelligent navigation [10]. However, developing algorithms capable of generating both feasible and smooth paths—while adapting to dynamic environments and considering the robot's kinematic constraints as well as the structured irregularity of orchards—remains a major challenge.

In recent years, the rapid advancements in artificial intelligence and autonomous driving technologies have significantly accelerated the deployment of agricultural robots, particularly in obstacle avoidance [11], target tracking [12–14], and path planning [15,16]. Among these aspects, path planning is a pivotal component in the autonomous operation of agricultural robots, as it directly impacts the safety, efficiency, and completeness of agricultural tasks [17]. A well-designed path planning strategy can effectively avoid densely distributed fruit trees, shrubs, and irregular obstacles in orchard environments, while also minimizing unnecessary turns and reverse maneuvers, thereby enhancing the continuity and stability of the operation. This is particularly crucial in spatially constrained environments such as orchards, where the quality of path planning determines whether the robot can maintain continuous and reliable operation. With the rapid development of intelligent algorithms, increasing research attention has been directed toward achieving global path optimization in terms of travel time, energy consumption, and motion smoothness, while ensuring safe obstacle avoidance [18,19]. Various methods have been applied to the path planning domain, such as deep learning-based path planning algorithms [20,21], bio-inspired planning strategies [22], and vision-based path recognition methods that extract navigation lines using machine vision [23–25]. These technologies continuously enhance the robot's adaptability to complex orchard environments.

Among the many path planning approaches, heuristic search algorithms such as A* [26], RRT* [27], and D* Lite [28] remain mainstream due to their strong path rationality and scalability, and they have been widely applied in agricultural robotics. While D* Lite offers efficient incremental replanning capabilities, it still suffers from issues such as insufficient path smoothness, redundant node expansion, and suboptimal heuristic functions, which limit its effectiveness in complex and dynamic environments. To address these problems, various improvements to D* Lite have been proposed. For example, Luo et al. [29] proposed an enhanced D* Lite algorithm for forest fire scenarios by introducing temperature perception and feasible segment filtering mechanisms, which improved the safety and responsiveness of UAVs in hazardous areas. However, this approach was mainly validated in 2D environments and did not fully consider stability and multi-constraint adaptability in complex 3D spaces. Jin et al. [30] integrated Conflict-Based Search (CBS) with the D* Lite algorithm to enhance path conflict detection and coordination capabilities for multi-robot systems in unknown dynamic environments. Although this approach performs well in obstacle avoidance, it incurs high computational costs, making it less suitable for real-time applications in resource-constrained orchard robot systems. Moreover, the focus of the method lies in task coordination rather than optimizing path quality and navigability. Gao et al. [31] combined global D* Lite with the local Dynamic Window Approach (DWA), reflecting a global–local hybrid planning strategy suitable for obstacle-dense areas. However, the specific features of orchard environments—such as structured row spacing, narrow aisles, and limited turning space—were not adequately

addressed. Li et al. [32] improved the D* Lite algorithm (hereinafter referred to as KD* Lite-B) by introducing a safety distance between the intelligent vehicle and obstacles. Subsequently, kinematic constraints of the vehicle were incorporated to expand the search directions and prevent path corners from exceeding the vehicle's steering maneuverability. The generated path was then optimized by removing redundant points to address issues of excessive curvature and unnecessary nodes, followed by smoothing with a third-order Bézier curve to produce a curvature-continuous trajectory. However, this method lacks a responsive mechanism for dynamic changes in structured orchard environments.

Despite the significant advances in path planning algorithms such as A, RRT, and D* Lite, their effectiveness in orchard environments remains limited due to complex terrain structures, narrow operational spaces, and localized environmental variations. Issues such as path deviation from the centerline, local congestion, low path smoothness, and constrained maneuverability persist, particularly when these algorithms are applied without customization. Moreover, existing enhancements to D* Lite often fail to fully incorporate the motion constraints faced by agricultural robots operating in structured yet irregular orchard layouts. To address these challenges, this study proposes a path planning method specifically designed for differential tracked robots in orchard environments. By abstracting discretely distributed fruit trees into a continuous narrow-passage model, the method enables a more structured representation of the workspace. On this basis, a hybrid path planning algorithm is developed by integrating an improved D* Lite framework, which enhances local feasibility and motion adaptability. The goal is to improve path smoothness, maneuverability, and overall navigation reliability under orchard-specific constraints. The effectiveness of the proposed method is validated through comprehensive simulations and field experiments.

2. Materials and Methods

2.1. D Lite Algorithm*

The D* Lite algorithm is an efficient pathfinding method designed for dynamic environments. It was proposed by Koenig and Likhachev in 2002 [33] as a simplified and optimized version of the original D* algorithm. D* Lite inherits the heuristic search framework of A*, while incorporating the incremental update capability of D*, allowing it to quickly repair paths when environmental changes occur without recalculating the entire global path. Due to its efficiency and adaptability, D* Lite has been widely applied in mobile robot navigation and dynamic obstacle avoidance scenarios.

This paper presents the key terms related to the D* Lite algorithm and their definitions in Table 1, with detailed explanations provided in the following chapters.

Table 1. Key terms related to the D* Lite algorithm and their definitions.

Term	Average Search Time (s)
$g(n)$	The actual path cost from the start node to node n, representing the estimated path cost to node n.
$h(n)$	Heuristic function, representing the estimated distance from node n to the goal node.
$rhs(n)$	The minimum estimated cost from a predecessor node to node n, representing the cost from a predecessor node to the goal node during path updates.
$key(n)$	The priority function used in the D* Lite algorithm to sort nodes in the OPEN list.
$c(s,n)$	The cost from node s to node n, typically the actual distance or cost between them.

The D* Lite algorithm introduces two key state variables, $g - value$ (current cost estimate) and $rhs - value$ (one-step lookahead cost), to represent the consistency of nodes and facilitate efficient dynamic path planning. It maintains a priority queue (OPEN list) to iteratively compute the shortest path in changing environments. During the initialization phase, the goal node is assigned $rhs(goal) = 0$, while all other nodes are initialized with $g(n) = rhs(n) = \infty$, and the goal is inserted into the OPEN list. In the iterative search process, the node with the smallest $key(n)$ in the OPEN list is repeatedly selected and updated based on its consistency status by modifying either its g-value or the rhs-values of its neighbors. Once the start node satisfies $g(start) = rhs(start)$ and has a key value not exceeding any node in the OPEN list, the search terminates and an optimal path can be extracted by tracing backward along g-values. When environmental changes occur—such as newly detected obstacles or map updates—only the costs of affected edges $c(s, n)$ need to be updated, and the corresponding nodes reinserted into the OPEN list to trigger local replanning, thereby avoiding full recomputation of the global path.

By combining the heuristic efficiency of A* with the incremental repair capability of D*, D* Lite achieves excellent computational efficiency and path quality, making it highly suitable for robotic navigation systems operating in dynamic environments.

*2.2. Hybrid A**

The traditional A* algorithm performs path search based on a regular grid map and primarily focuses on the two-dimensional spatial domain (x, y). However, it encounters significant limitations when applied to mobile robot path planning problems that involve heading constraints or kinematic restrictions. To address these issues, the Hybrid A* algorithm [34] was developed as a continuous-space extension of the A* framework. While retaining the heuristic properties of A*, Hybrid A* introduces a heading angle variable and expands the search space into a three-dimensional configuration space (x, y, θ), thereby enabling the generation of paths that are feasible under nonholonomic constraints.

The core idea of Hybrid A* lies in combining a discrete motion primitive set with continuous path evaluation. During the expansion of each node, the algorithm no longer performs simple movements along grid directions. Instead, a set of feasible trajectory segments is generated in real time based on the robot's kinematic model (e.g., differential drive), ensuring that each expanded motion respects the physical feasibility of the system. Each node state is composed not only of a 2D position (x, y) but also of a discretized heading angle θ, selected from a predefined set of orientations:

$$N_\theta = \{\theta_1, \theta_2, \ldots, \theta_{N_\theta}\}, \theta_i = \frac{2\pi(i-1)}{N_\theta}, i = 1,2, \ldots, N_\theta \quad (1)$$

The robot's current continuous heading angle θ is mapped to the nearest discrete angle in the set N_θ. Based on the current orientation and the direction to the target, the algorithm selects an appropriate motion primitive (e.g., forward, left turn, right turn, reverse) to generate the next state.

Compared with traditional A*, which only supports movement in horizontal, vertical, and diagonal directions, Hybrid A* is capable of generating smoother paths with stronger physical feasibility and continuous curvature. This makes it particularly suitable for mobile robot systems that must account for turning radius constraints and smooth navigation requirements.

2.3. Kinematic Model of Differential Tracked Robot

To meet the path planning requirements of an orchard spraying robot equipped with a differential tracked chassis, the kinematic model of the robot is established, as illustrated in Figure 1.

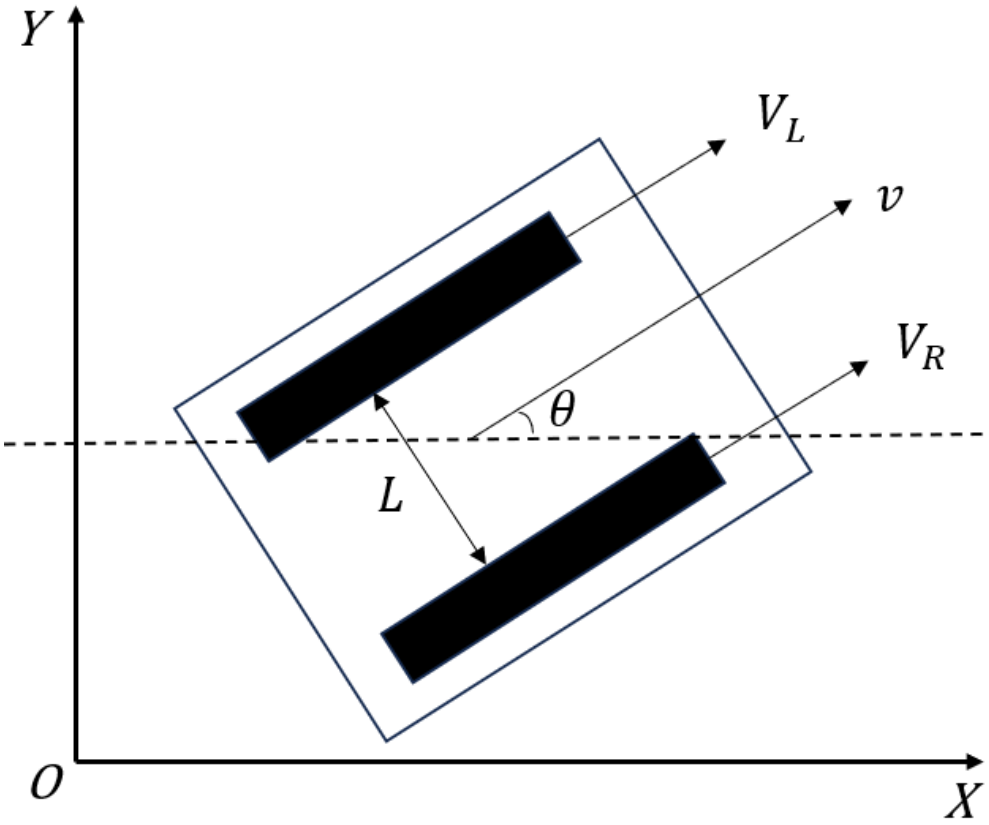


Figure 1. Kinematic model of the orchard spraying robot: V_L represents the left wheel speed of the sprayer, V_R represents the right wheel's forward speed, θ is the robot's heading angle, and L is the distance between the two driving wheels, v is the linear velocity of the robot, and the dashed line represents a line parallel to the X-axis.

According to the differential wheel drive principle, the robot's overall linear velocity v and angular velocity ω can be expressed by the left and right wheel speeds as follows:

$$v = \frac{V_R + V_L}{2} \tag{2}$$

$$\omega = \frac{V_R - V_L}{L} \tag{3}$$

When the robot is turning at a non-zero velocity, its instantaneous center of rotation lies on the extension of the line connecting the left and right wheels. The turning radius R and steering curvature κ are given by the following equation:

$$R = \frac{L}{2} \times \frac{V_R + V_L}{V_R - V_L} \tag{4}$$

$$\kappa = \frac{1}{R} = \frac{2}{L} \times \frac{V_R - V_L}{V_R + V_L} \tag{5}$$

Assuming the robot's current pose is $[x, y, \theta]$, the velocity components of the robot's center point in the global coordinate system are as follows:

$$\dot{x} = v \times cos(\theta) = \frac{V_R + V_L}{2} \times cos(\theta) \quad (6)$$

$$\dot{y} = v \times sin(\theta) = \frac{V_R + V_L}{2} \times sin(\theta) \quad (7)$$

$$\dot{\theta} = \omega = \frac{V_R - V_L}{L} \quad (8)$$

2.4. A Hybrid Path Planning Algorithm Based on Improved D Lite*

2.4.1. Initial Path Planning Based on Voronoi Diagram

To accommodate the structural characteristics of orchard environments, this study makes the following modeling assumptions: the orchard consists of regularly arranged fruit trees, forming multiple narrow longitudinal corridors; the operation robot is expected to move along the center lines between the tree rows as much as possible to ensure operational stability and complete coverage.

Due to the regularity of fruit tree planting, the width of the passages between tree rows may be influenced by the tree density, with these passages typically being less than 2 m wide. Thus, they can be defined as "narrow rows". In such confined spaces, the robot needs to optimize its path planning according to the operational requirements (e.g., precise spraying), avoiding deviation from the centerline and ensuring the completeness and stability of the spraying coverage.

In such scenarios, traditional path planning algorithms based on D* or A* often adopt Euclidean or Manhattan distance as the heuristic function, which lacks structural awareness in estimating the goal distance. Particularly in elongated, regular, yet constrained spaces, these methods are prone to issues such as path deviation from the center line, distortion of search direction, and redundant node expansions. To enhance the structural adaptability of the path, this study introduces the Voronoi diagram [35] as a structure-aware path guidance tool, constructing an environmental skeleton path to mitigate the interference of narrow corridors on the heuristic function.

The Voronoi diagram is a widely used spatial partitioning method in computational geometry, primarily used to describe the distance relationships between a set of points (referred to as "sites" or "generating points") and other points in the space. The core idea is to divide the space into multiple regions, each containing all the points that are closest to a specific generating point. These regions are called Voronoi cells, and each generating point corresponds to one cell. Every point in space is assigned to its nearest generating point. Let us assume a given set of generating points:

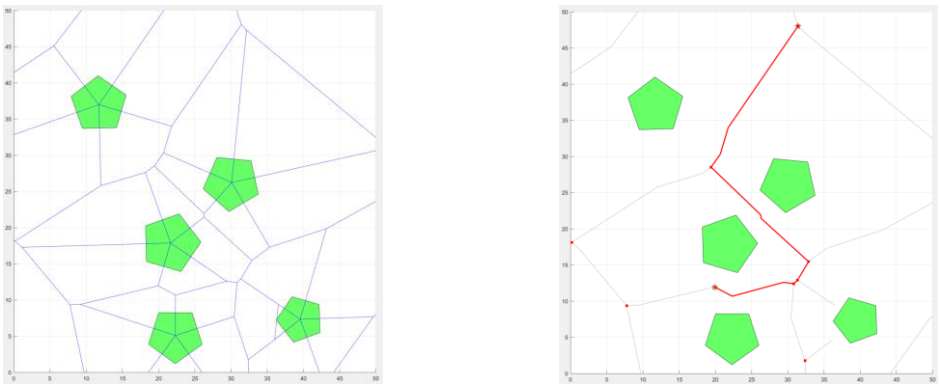
$$O = \{o_1, o_2, \ldots, o_k\} \quad (9)$$

Each point o_i is referred to as a site. For any point p in space, if the distance between p and a particular site o_i is closer than the distance between p and any other site, then p belongs to the Voronoi cell $V(o_i)$ of site o_i. The mathematical expression is as follows:

$$V(p) = \left\{ p \in R^2 \,\middle|\, ||p - o_i|| \leq ||p - o_j||, \forall j \neq i \right\} \quad (10)$$

Here, $p \in R^2$ indicates that point p is located in the two-dimensional space R^2, where R^2 represents the plane formed by all two-dimensional real number coordinates.

The skeleton edges of the Voronoi diagram lie in regions equidistant from multiple obstacle points. These regions are farthest from obstacles and represent the safest areas in the environment, thus serving as structural guidance lines for path planning (Figure 2).



(**a**) (**b**)

Figure 2. Voronoi diagram and initial path planning illustration: (**a**) original Voronoi diagram; (**b**) path planning based on Voronoi diagram. The green geometric shapes in the figure represent obstacles in the simulation environment, the blue segments represent the Voronoi diagram edges, and the red segments represent the final reference path found.

2.4.2. Cost Calculation of Nodes in the Improved D* Algorithm

The cost function form of D* Lite is similar to that of the A* algorithm, with the basic cost function defined as follows:

$$f(n) = g(n) + h(n) \tag{11}$$

where n denotes the current node to be expanded, $g(n)$ represents the current cost from the start node to node n, and $h(n)$ is the heuristic estimate of the cost from node n to the goal node, typically computed using Euclidean or Manhattan distance.

In the D* Lite algorithm, $rhs(n)$ represents the minimum estimated cost to reach node n from a predecessor node. Specifically, $rhs(n)$ is determined by considering the path cost $g(s)$ from the start node to the predecessor node s, and the cost $c(s,n)$ from the predecessor node s to the target node n. Therefore, the calculation formula for $rhs(n)$ is as follows:

$$rhs(n) = \min_{s \in Pred(n)} [g(s) + c(s,n)] \tag{12}$$

where $Pred(n)$ denotes the set of all predecessor nodes of n, and $c(s,n)$ is the cost of moving from node s to node n.

D* Lite updates paths by checking the consistency condition of nodes. If $g(n) = rhs(n)$, node n is consistent; if $g(n) > rhs(n)$, node n is overconsistent and needs to propagate better path costs to its successors; if $g(n) < rhs(n)$, node n is inconsistent and requires back-propagation to predecessors for correction.

To efficiently select the node to expand, D* Lite introduces a dual-key priority function $key(n)$ to sort the nodes in the OPEN list, defined as follows:

$$key(n) = [min\big(g(n), rhs(n)\big) + h(start, n), min(g(n), rhs(n))] \tag{13}$$

The first key, $min\big(g(n), rhs(n)\big) + h(start, n)$, combines the local cost estimate and the heuristic function to ensure the efficiency of global search. The second key, $min(g(n), rhs(n))$, serves as the secondary sorting criterion based on local cost. This function takes into account both the estimated path cost from the start node to the current node and the consistency status of the node, which is used to drive the heuristic search.

Actual Cost Calculation of Nodes

Considering the practical operating conditions of orchard robots, it is required that the robot travels as closely as possible along the centerline of the tree rows while

minimizing unnecessary steering during motion. Therefore, in addition to the conventional node-to-node distance cost, the proposed algorithm introduces a centerline deviation cost, a steering cost, and a motion inertia cost. These cost terms jointly constrain the robot's behavior during the node expansion process, guiding the generation of more reasonable and safer navigation paths.

The actual cost calculation of nodes needs to incorporate the real working conditions of the orchard robot. The robot is required to travel as close as possible to the centerline of the tree rows while minimizing unnecessary turns during movement. Therefore, besides considering the distance cost between nodes, this algorithm introduces additional costs for deviation from the tree row centerline, steering maneuvers, and driving inertia. These combined constraints guide the robot's behavior during node search, facilitating the generation of more reasonable and safer navigation paths.

The Euclidean distance cost is used to measure the straight-line distance between the current node $s(x_s, y_s)$ and the node to be expanded $n(x_n, y_n)$. This is the most fundamental cost metric, ensuring the overall compactness of the path and helping to control the total travel distance. The cost is calculated as follows:

$$d_{euclidean}(s,n) = \sqrt{(x_n - x_s)^2 + (y_n - y_s)^2} \tag{14}$$

The centerline deviation cost reflects the distance between node n and the structural centerline of the orchard row. It serves to guide the path closer to the central axis of the operating area, thereby reducing the risk of edge-clinging paths or dead-end incursions. The cost function is defined as follows:

$$d_{\text{center}}(n) = \| (x_n, y_n) - (x_c, y_c) \| \tag{15}$$

where (x_c, y_c) denotes the point on the orchard row centerline generated by the Voronoi diagram that corresponds to node n.

The turning cost is used to quantify the degree of directional change along the path. It restricts abrupt turns between nodes, encouraging angular continuity in the trajectory. This helps reduce control fluctuations caused by frequent steering. The cost function is defined as follows:

$$c_{\text{turn}}(s,n) = | \theta_{sn} - \theta_s | \tag{16}$$

where θ_{sn} represents the heading angle from node s to node n, and θ_s is the robot's previous heading angle upon arriving at node s.

$$\theta_{sn} = arctan2(y_n - y_s, x_n - x_s) \tag{17}$$

The path inertia cost is used to evaluate the directional consistency of the trajectory. It penalizes expansion actions that deviate from the direction of the previous path segment, reflecting the inertial behavior of the robot during movement. This cost effectively prevents frequent U-turns or sharp turns in the path, thereby improving trajectory continuity and executability. The cost function is defined as follows:

$$c_{\text{inertia}}(s,n) = 1 - \frac{\vec{v}_{sn} \cdot \vec{v}_s}{||\vec{v}_{sn}|| \times ||\vec{v}_s||} \tag{18}$$

The direction vector from the current node s to the successor node n, and the historical direction vector from the previous node s_{prev} to the current node s are defined as follows:

$$\vec{v}_{sn} = [x_n - x_s, y_n - y_s] \tag{19}$$

$$\vec{v}_s = [x_s - x_{prev}, y_s - y_{prev}] \tag{20}$$

To achieve a balance among multiple objectives, this paper combines the above four basic cost components through weighted summation, constructing the final actual node cost function as follows:

$$c(s,n) = d_{euclidean}(s,n) + \lambda_1 \times d_{center}(n) + \lambda_2 \times c_{turn}(s,n) + \lambda_3 \times c_{inertia}(s,n) \quad (21)$$

In this study, we normalized multiple cost factors in the path planning process to ensure fairness and rationality among the cost terms during optimization. Specifically, three main weight coefficients are defined: λ_1, λ_2, and λ_3, corresponding to the penalty factors for structural deviation, steering changes, and inertia deviation, respectively. First, λ_1 (structural deviation weight) is set to 2.0 to strengthen the alignment of the path with the row centerline, reducing the risk of deviation from the centerline, thereby ensuring the structural rationality and operational precision of the path. Second, λ_2 (steering cost weight) is set to 0.2, aiming to limit excessive direction changes along the path, ensuring smoothness and continuity while avoiding sharp turns in the path. Finally, λ_3 (inertia deviation weight) is set to 0.3, designed to ensure the consistency of the path's direction, avoid frequent reversals or sharp turns, and improve the executability and continuity of the path.

Heuristic Cost Calculation of Nodes

In heuristic search-based path planning algorithms, the heuristic function $h(n)$ represents the estimated cost from the current node n to the goal node. Its design directly impacts the quality of the search path and the efficiency of the search process. When $h(n)$ is always less than or equal to the actual cost, the algorithm guarantees finding the optimal path, but it often expands a large number of nodes, resulting in lower search efficiency. Conversely, if $h(n)$ overestimates the actual cost, the search speed can be accelerated, but there is a risk of obtaining suboptimal paths.

To meet the demands of path planning in complex structured orchard environments, and considering the kinematic characteristics of differential tracked robots, the heuristic function in this work incorporates not only the distance to the goal but also integrates factors such as path direction consistency, deviation from the centerline, and obstacle distribution during node expansion. This approach more effectively guides the search process, improving both path feasibility and smoothness.

The goal distance cost measures the Euclidean distance between the expanded node $n(x_n, y_n)$ and the goal node $g(x_g, y_g)$. It is the most fundamental heuristic estimate, ensuring the convergence of the path and serving as a basic indicator of path optimality. The cost is calculated as follows:

$$d(n,g) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2} \quad (22)$$

To incorporate the direction change constraints of differential tracked robots, the path planning encourages alignment between the path direction and the target direction. This is measured by the cosine difference in the angle between the direction from the current node s to the expanded node n and the direction from node n to the goal. This cost term penalizes sharp turns and frequent reversals in direction along the path, thereby promoting smoother and more controllable trajectories. The cost function is expressed as follows:

$$c_{nonholonomic}(n) = 1 - \cos(\theta_{sn} - \theta_{ng}) \quad (23)$$

The direction from the expanded node to the target is defined as follows:

$$\theta_{ng} = arctan2(y_g - y_n, x_g - x_n) \quad (24)$$

The obstacle density cost takes into account the dense distribution of obstacles in the orchard environment, requiring the path planning to keep as far away from obstacles as possible. This term measures the density of obstacles around the expanded node n, effectively penalizing paths that approach obstacle boundaries and thereby enhancing the safety margin of the path. The cost function is defined as follows:

$$G_{obstacle}(n) = \sum_{i=1}^{N} \frac{1}{||(x_n, y_n) - (x_i, y_i)||^2 + \epsilon} \quad (25)$$

Here, (x_i, y_i) represents the positions of local obstacle points, and ϵ is a small positive constant to prevent division by zero.

The above cost terms are combined with weights to form the final heuristic function used for node expansion evaluation as follows:

$$h(n) = d(n, goal) + \alpha \times d_{center}(n) + \beta \times c_{nonholonomic}(n) + \gamma \times G_{obstacle}(n) \quad (26)$$

After normalizing the above factors using the min–max normalization method, α, β, and γ serve as weight coefficients, ensuring fairness and rationality among the cost terms during optimization. Specifically, in this study, we set the centerline offset heuristic factor (α = 0.8), the non-holonomic motion constraint penalty factor (β = 0.2), and the obstacle density penalty factor (γ = 0.4). These factors were chosen to ensure the path's safety, rationality, and feasibility. The setting of α strengthens the alignment of the path with the row centerline, ensuring that the path more accurately follows the centerline. β ensures the feasibility of the path under the differential drive robot's kinematic constraints, avoiding path complexity. γ reduces the risk of the path approaching obstacles by considering their density, thus enhancing the safety of the path. After normalization, α, β, and γ as weight coefficients ensure fairness and rationality among the cost terms during the optimization process, enabling the robot to achieve efficient and safe path planning in the complex orchard environment.

2.4.3. Hybrid A* Optimization for Narrow Sections

When a path passes through narrow regions, the combination of spatial constraints and kinematic limitations often leads to issues such as discontinuity, poor feasibility, or excessively small turning radii. These problems make it difficult for a differentially tracked robot to traverse the area smoothly. To address this, the Hybrid A* algorithm is applied to locally replan the initial path within narrow sections, generating smooth curved paths that better comply with kinematic constraints.

When a narrow passage is detected, the system automatically activates the local path replanning mechanism. First, based on a minimum safe distance threshold d_{min} between the path and surrounding obstacles, the algorithm extracts continuous segments of the global path that are too close to obstacles. These segments are marked as narrow path sections, denoted by $P_{narrow} = \{p_s, \ldots, p_t\}$. Next, a distance δ is extended in both directions along the path to form an expanded local replanning region $R_{local} = \{p_s - \delta, \ldots, p_t + \delta\}$. Within this region, the Hybrid A* algorithm is used for local path replanning. The local starting state is defined as $S_{start} = (x_s, y_s, \theta_s)$ and the goal state as $S_{goal} = (x_t, y_t, \theta_t)$, where θ represents the direction of the path tangent at a point. It is calculated using the following equation:

$$\theta_i = atan2(y_{i+1} - y_i, x_{i+1} - x_i) \quad (27)$$

Within the defined local region, the Hybrid A* algorithm performs path planning based on the updated obstacle distribution and the Voronoi center line. By incorporating a continuous state space (x, y, θ) and differential drive motion primitives (including forward, turning, and reverse motions), the algorithm generates trajectories that satisfy

nonholonomic motion constraints, ensuring smooth curvature and dynamic feasibility. The newly generated trajectory then replaces the original narrow section of the path, improving local path feasibility and enhancing the overall continuity of the global path.

2.4.4. Path Smoothing Optimization

Within the hybrid path planning framework, although the improved D* Lite algorithm—integrated with Voronoi diagrams and Hybrid A*—is capable of generating feasible paths, the resulting trajectories may still exhibit local irregularities in narrow passages or complex regions, such as unsmooth segments or unnatural turning behaviors. To further enhance path execution continuity and driving comfort, this paper introduces a cost- and curvature-constrained path smoothing optimization method, applied after the initial path generation by D* Lite.

Assume the final path consists of N nodes with a position sequence $X_i = (x_i, y_i), i \in [1, N]$. The displacement vector between nodes is defined as follows:

$$\Delta X_i = X_i - X_{i-1} \tag{28}$$

The change in heading angle at node i is defined as follows:

$$\Delta\Phi_i = \left|\tan^{-1}\frac{\Delta y_{i+1}}{\Delta x_{i+1}} - \tan^{-1}\frac{\Delta y_i}{\Delta x_i}\right| \tag{29}$$

A global smoothing process is then applied to the path by minimizing the following objective function:

$$F = w_c\sum_{i=1}^{N}(cost(X_i))^2 + w_s\sum_{i=1}^{N}||\Delta X_{i+1} - \Delta X_i||^2 + +w_k\sum_{i=1}^{N}\sigma_\kappa(\frac{\Delta\Phi_i}{||\Delta X_i||} - \kappa_{max}) \tag{30}$$

Here, w_c, w_s and w_k are the weight coefficients for the cost, smoothness, and curvature terms, respectively. $cost(X_i)$ denotes the Voronoi cost at node, encouraging the path to stay close to the central safe region. The second term penalizes sharp direction changes between adjacent path segments to improve smoothness. The third term enforces curvature constraints to ensure that the local curvature does not exceed the robot's maximum allowable curvature κ_{max}. The curvature penalty function σ_κ is defined as follows:

$$\sigma_\kappa(x) = \begin{cases} x^2, x > 0 \\ 0\ , x \le 0 \end{cases} \tag{31}$$

By minimizing this objective function, the resulting path adheres closely to the Voronoi center line while satisfying the kinematic constraints of a differential tracked vehicle. This approach enhances overall path smoothness and feasibility, and effectively adapts to trajectory optimization requirements following dynamic updates or local replanning in the D* Lite algorithm.

2.5. Improved D Lite Algorithm Process*

This paper addresses the path planning requirements for orchard robots navigating narrow tree rows by proposing a hybrid path planning algorithm based on an improved D* Lite algorithm, as shown in Figure 3. The method first constructs a two-dimensional grid map of the orchard and extracts the centerlines of the tree rows using a Voronoi diagram, guiding the global path to closely follow the central axis of the working corridors. During the initial path planning phase, the improved D* Lite node cost function incorporates multiple cost components, including centerline deviation, turning penalties, and nonholonomic constraints, to enhance the feasibility and structural rationality of the path. Additionally, a composite heuristic function integrating structural information and obstacle gradients is designed to accelerate path searching and reduce redundant node expansions. To address the issue of the algorithm potentially running indefinitely due to

continuous changes in the environment, this paper introduces stopping criteria based on a maximum iteration count and stability check during the path planning process.

After global path planning is completed, the system automatically detects narrow segments along the path and expands both ends of each segment by a fixed distance to define the local replanning region for Hybrid A*. Within each local section, the modified Hybrid A* algorithm performs local path optimization. However, the replanned segments may still exhibit local irregularities, such as abrupt turns or discontinuities. To address this, a gradient descent method is applied to further smooth the path, ultimately forming a complete, executable, and continuous global trajectory.

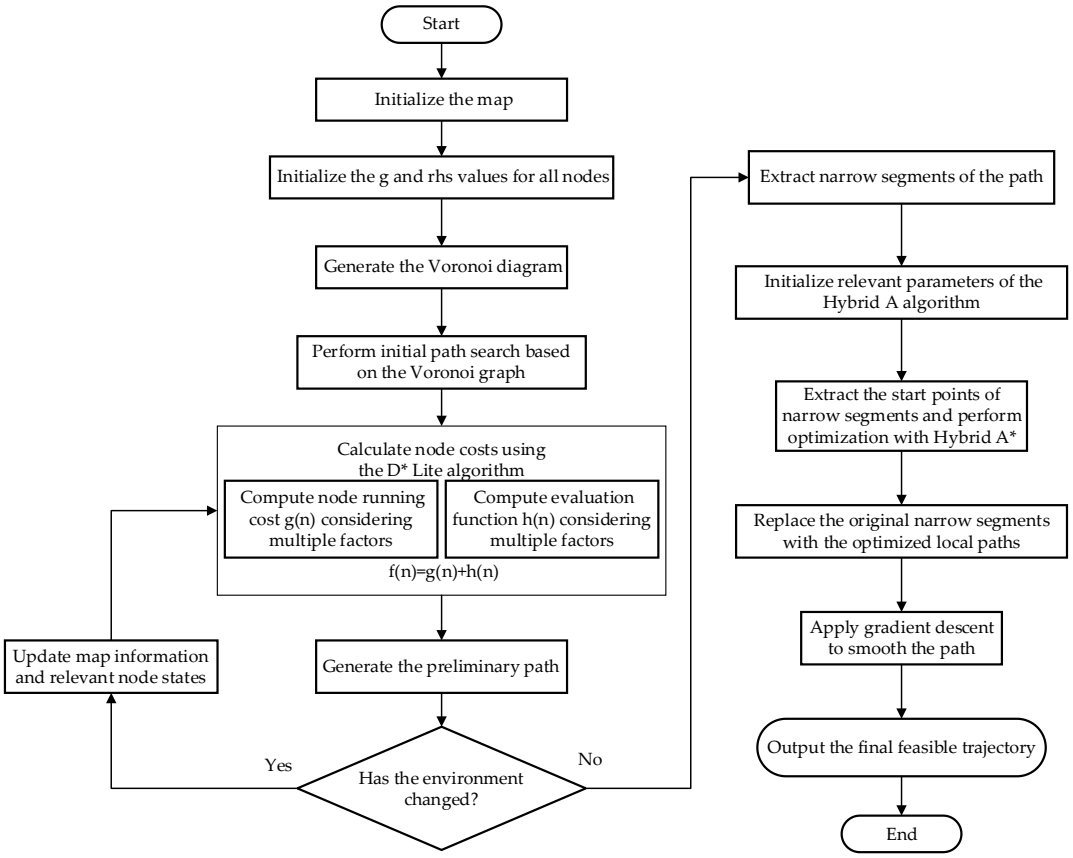


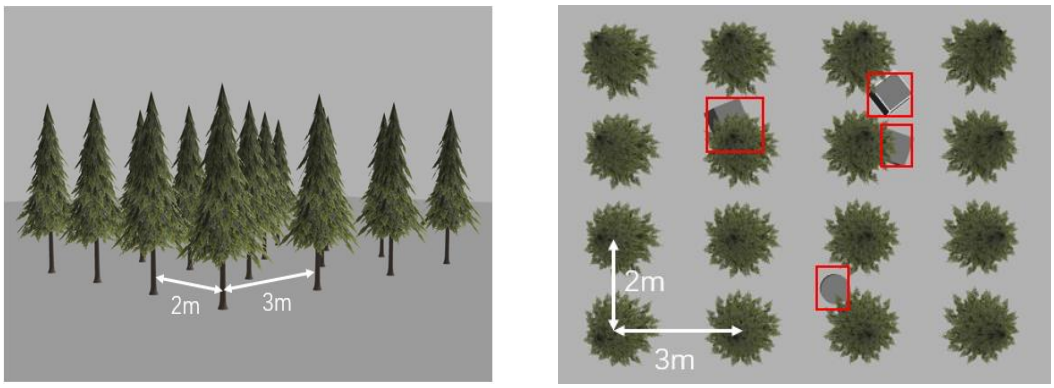
Figure 3. Flowchart of the hybrid path planning algorithm based on improved D* Lite.

3. Results

3.1. Simulation Results

3.1.1. Simulation Environment Setup

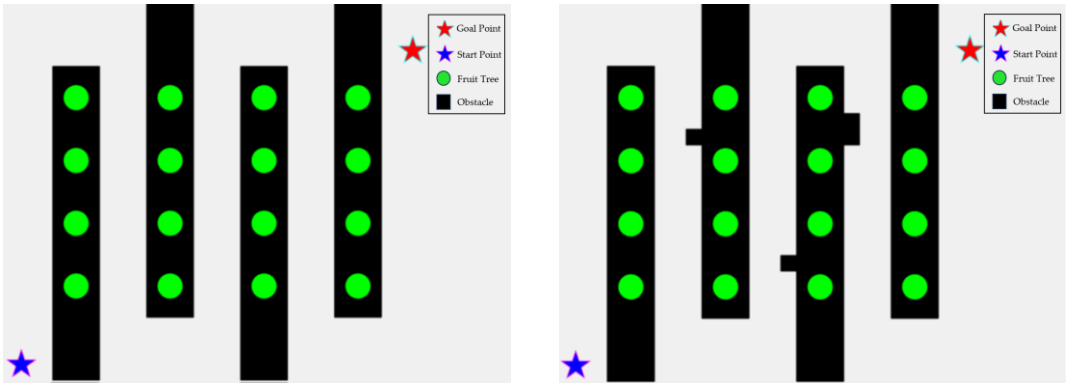
To verify the feasibility and effectiveness of the proposed hybrid path planning algorithm based on the improved D*Lite algorithm, simulation experiments were conducted using the RViz and Gazebo platforms within the Robot Operating System 2 (ROS2) [36]. A standardized orchard simulation environment was established as shown in Figure 4. The figure shows that the row spacing between plant lines in the scene is 3 m, and the intra-row spacing between individual plants is 2 m. Figure 4b illustrates the orchard environment with added obstacles, including cuboid and cylindrical obstacles. The dimensions of the cuboid obstacles are approximately 1 m × 1 m and 0.8 m × 0.8 m, representing facilities in the orchard such as storage boxes, irrigation equipment, etc. The cylindrical obstacles have a diameter of 0.8 m, simulating weeds or other small obstacles.



(**a**) (**b**)

Figure 4. Simulated orchard environment: (**a**) original orchard environment; (**b**) orchard environment with added obstacles, where the red square outlines the obstacles.

Based on the simulated orchard environment, a navigation map for orchard simulation experiments was established, as shown in Figure 5.



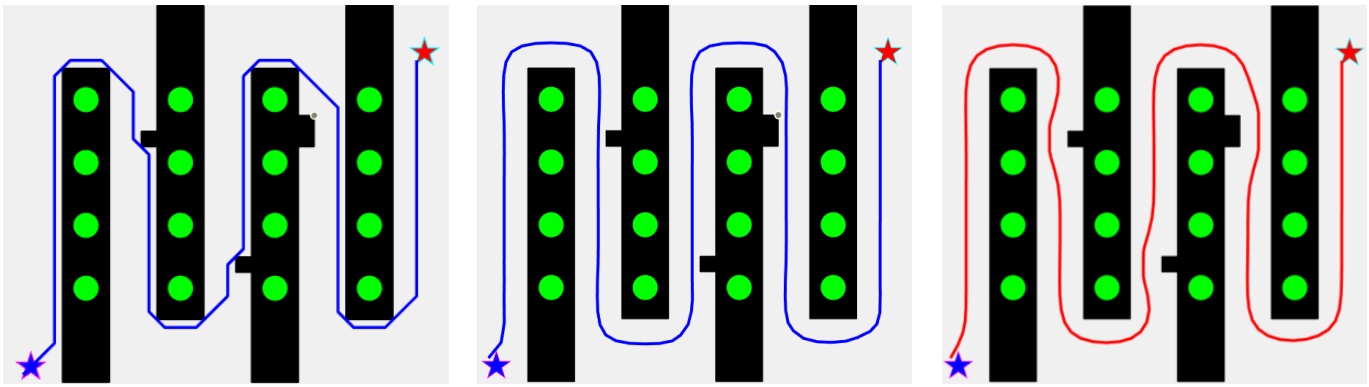
(**a**) (**b**)

Figure 5. Simulation experiment navigation maps: (**a**) original orchard navigation map; (**b**) orchard navigation map after introducing additional obstacles. The blue star-shaped symbol represents the starting point, the red star-shaped symbol indicates the goal point, the green areas denote the distribution of fruit trees, and the black areas correspond to the expanded obstacles. The same symbols are used in all subsequent figures.

The testing platform is a self-developed tracked orchard spraying robot, which features excellent obstacle-crossing capability and stable differential drive control. To ensure high consistency between simulation and physical tests, the robot's key physical parameters and control characteristics were carefully measured. The overall dimensions of the orchard spraying robot are 1.13 m (length) × 0.86 m (width) × 1.97 m (height), with a maximum speed of 1.5 m/s.

3.1.2. Performance Comparison of the Algorithm

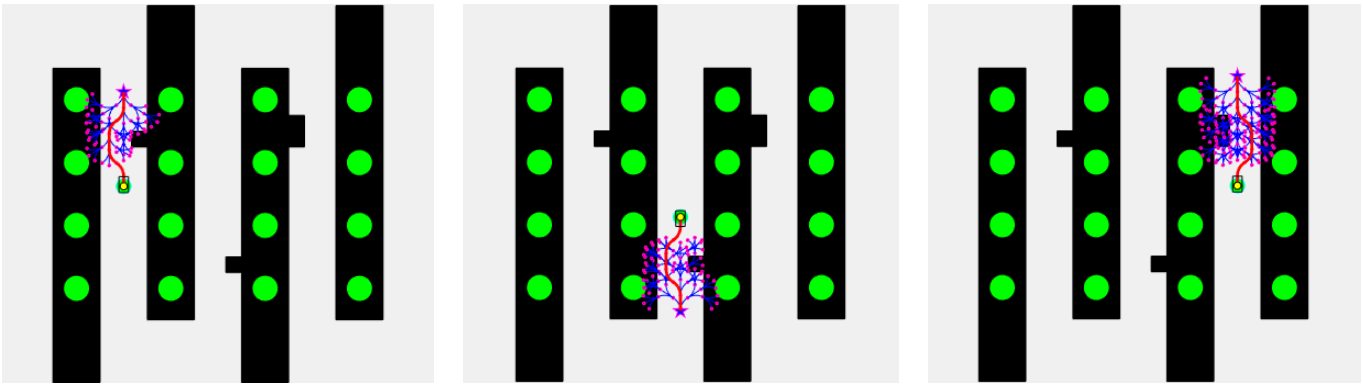
To validate the comprehensive performance of the proposed hybrid path planning algorithm in narrow orchard environments, comparative analyses were conducted against the traditional D* Lite algorithm and the KD* Lite-B algorithm. The simulation experiments were carried out on a simulated orchard map, and the corresponding path planning results are presented in Figure 6.



(**a**) (**b**) (**c**)

Figure 6. Simulation results of path planning in orchard scenario: (**a**) traditional D* Lite algorithm with blue lines; (**b**) KD* Lite-B algorithm with blue lines; (**c**) proposed hybrid algorithm based on improved D* Lite with red lines.

After the preliminary path planning of the hybrid algorithm is completed, the system automatically identifies narrow passage regions along the path. It then extends both ends of each narrow segment in the direction of travel to define local replanning areas. Subsequently, an improved Hybrid A* algorithm is employed to perform local trajectory optimization within these segments, generating continuous and feasible curve segments. The results of local replanning are shown in Figure 7. Once all narrow segments are replaced, a gradient descent method is applied to further smooth the entire path and enhance its continuity and controllability, ultimately producing the final feasible global trajectory, as illustrated in Figure 6c.

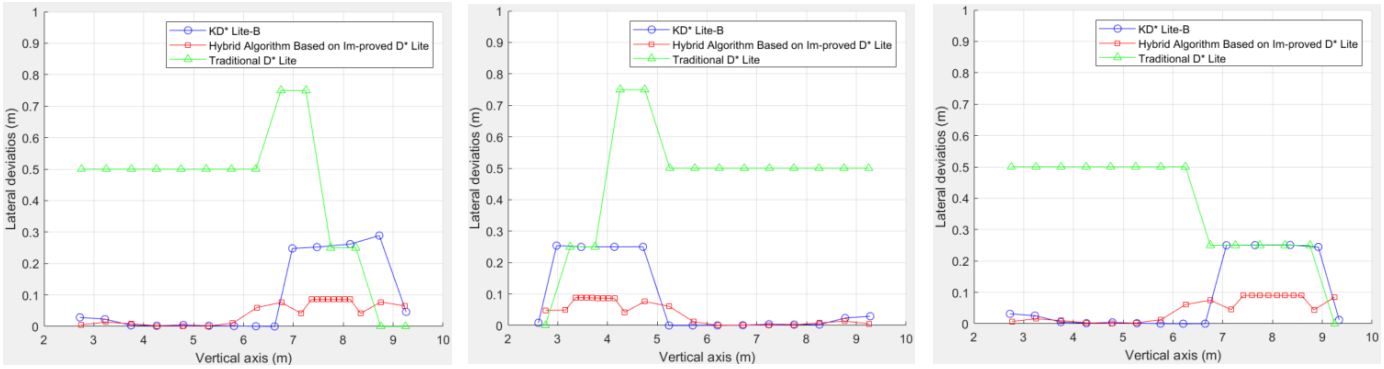


(**a**) (**b**) (**c**)

Figure 7. The simulation scenario path local re-planning test results: (a) re-planning result for narrow section 1; (b) re-planning result for narrow section 2; (c) re-planning result for narrow section 3. The blue curve represents the exploration path of the Hybrid A* algorithm during the search process, while the red line represents the optimized path, which is the re-planning result, the yellow circle marks the endpoint of the local re-planning.

Considering that orchard robots primarily operate between tree rows, this study compares the lateral deviations of trajectories generated by three different path planning algorithms with respect to the centerlines of inter-row spaces, excluding row-changing and turning maneuvers. As shown in Figure 8, the lateral deviations of the traditional D* Lite, KD* Lite-B, and the proposed improved algorithm are illustrated. The results indicate that the improved algorithm consistently exhibits smaller lateral deviations across all inter-row sections. In particular, in narrow passages, the trajectory closely aligns with the

row centerline, demonstrating superior overall performance in terms of path accuracy and stability compared to the traditional algorithms.



(**a**) (**b**) (**c**)

Figure 8. Comparison of lateral deviations from the tree row centerline in the simulation scenario: (**a**) first inter-row space; (**b**) second inter-row space; (**c**) third inter-row space.

The deviations of the global paths generated by different algorithms from the crop row centerline are shown in Table 2. The data indicate that the proposed hybrid algorithm based on the improved D* Lite consistently yields smaller lateral deviations across all inter-row sections compared to the traditional D* Lite and KD* Lite-B algorithms. In the first inter-row space, the hybrid algorithm achieves an average deviation of 0.0520 m, representing a substantial reduction of approximately 87.9% compared to the 0.4286 m deviation of traditional D* Lite, and also lower than the 0.0831 m deviation of KD* Lite-B. In the second inter-row space, the deviation is reduced from 0.4643 m (traditional D* Lite) to 0.0486 m, a decrease of 89.5%, and is also 36.7% lower than that of KD* Lite-B (0.0769 m). For the third inter-row space, the hybrid algorithm achieves a deviation of 0.0541 m, reducing the error by 85.6% compared to traditional D* Lite (0.3750 m), and outperforming KD* Lite-B (0.0771 m) by approximately 29.8%. Overall, the proposed algorithm demonstrates superior accuracy and stability in maintaining the path close to the crop row centerline, significantly enhancing both the rationality and safety of the planned path.

Table 2. Comparison of deviations from the crop row centerline in the simulation scenario.

Algorithm	First Inter-Row Space (m)	Second Inter-Row Space (m)	Third Inter-Row Space (m)
Traditional D* Lite	0.4286	0.4643	0.3750
KD* Lite-B	0.0831	0.0769	0.0771
Hybrid Algorithm Based on Improved D* Lite	0.0520	0.0486	0.0541

To systematically evaluate the performance of different path planning algorithms in structured orchard environments, this study defines key metrics from four perspectives: path centrality, safety, path length, and planning efficiency. Comparative experiments were conducted in a simulated orchard scenario for the traditional D* Lite, KD* Lite-B, and the proposed hybrid algorithm based on an improved D* Lite. The results are summarized in Table 3.

In terms of path centrality, the traditional D* Lite algorithm, due to the lack of structured path guidance, causes the path to deviate significantly from the centerline of the orchard rows, with an average deviation of 0.4226 m, which affects operational continuity and accuracy. In contrast, the hybrid algorithm based on the improved D* Lite introduces a "centerline deviation penalty" and combines Voronoi guidance, effectively keeping the path close to the centerline of the orchard rows. The average deviation is reduced to 0.0516

m, a reduction of approximately 87.8% compared to the traditional algorithm, and is also better than KD* Lite-B*'s 0.0790 m, significantly improving the structural adaptability of the path.

In terms of safety assessment, the number of hazardous nodes near the obstacle boundaries along the path is used as an indicator. The traditional D* Lite path contains 74 hazardous nodes, posing a higher collision risk. The hybrid algorithm identifies narrow areas and locally applies Hybrid A* for path replanning, generating a safe path that meets the kinematic constraints of the robot. The number of hazardous nodes is reduced to 3, a decrease of 57.1% compared to KD* Lite-B's 7 and 95.9% compared to the traditional algorithm's 74, greatly improving the safety of the path.

In terms of path length, the hybrid algorithm plans a path of 55.26 m, which is slightly shorter than KD* Lite-B's 56.35 m. Although it is slightly longer than the traditional algorithm's 51.52 m, the path is smoother and more continuous, with better executability. Although the hybrid algorithm has an average search time of 12.416 s, higher than the traditional D* Lite's 8.578 s and KD* Lite-B's 6.898 s, this is primarily due to the additional local path optimization and curve processing steps. However, the quality, safety, and execution stability of the path have significantly improved, which helps reduce the frequency of trajectory corrections and control costs during actual operations, thereby improving the system's overall operational efficiency and robustness.

Table 3. Performance comparison of path planning algorithms in a simulated orchard environment.

Algorithm	Average Search Time (s)	Number of Dangerous Nodes (Count)	Path Length (m)	Overall Mean Lateral Deviation from Centerline (m)
Traditional D* Lite	8.578	70	51.52	0.4226
KD* Lite-B	6.898	7	56.35	0.0790
Hybrid Algorithm Based on Improved D* Lite	12.416	3	55.26	0.0516

3.2. Field Experiments and Results

3.2.1. Experimental Platform

The algorithm proposed in this paper is validated using an orchard spraying robot with a tracked chassis as the experimental platform, as shown in Figure 9.

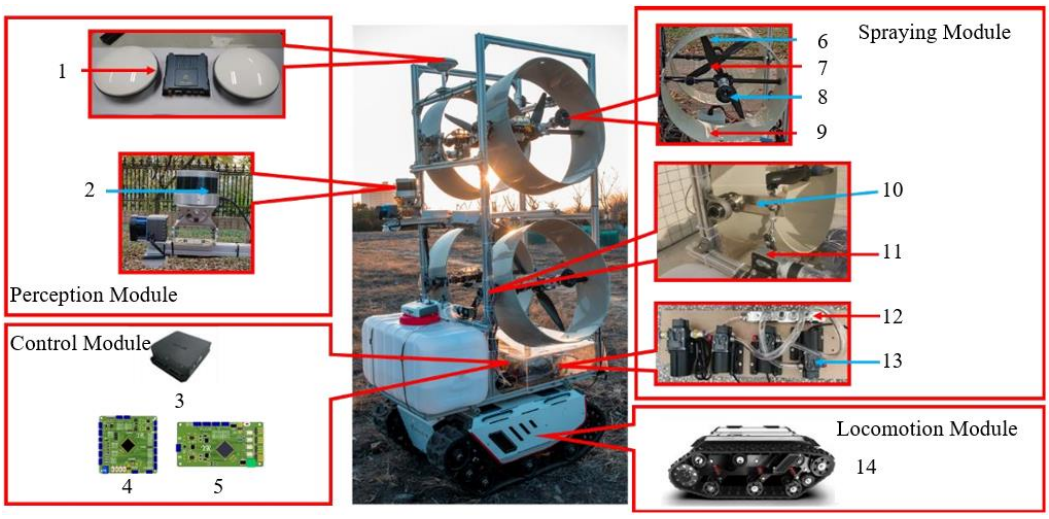


Figure 9. Schematic diagram of the orchard spraying robot. Perception module: 1. BeiDou integrated navigation system; 2. Three-Dimensional LiDAR; control module; 3. central processing unit; 4, 5. embedded system; spraying module: 6. blade; 7. blower; 8. centrifugal nozzle; 9. duct; 10. crank-

rocker mechanism; 11. worm gear reducer motor; 12. flow divider; 13. water distribution pump; locomotion module: 14. tracked chassis.

In this study, the central computer is equipped with the Ubuntu 22.04 LTS operating system and developed based on Robot Operating System 2 (ROS2, Foxy Fitzroy) [37]. The central controller uses an Intel 8th generation Core i7-8565U processor, which provides excellent computational power and low power consumption, capable of supporting high-intensity computational tasks such as 3D LiDAR point cloud processing, SLAM mapping, and path planning. The system is equipped with 8 GB of RAM to ensure stable multi-task processing and efficient real-time computation. The robot's position information in the global coordinate system is first obtained via a combined navigation module. The relative coordinates of the orchard trees and the robot are acquired using the 3D LiDAR, and after coordinate transformation, the global coordinates of the trees are calculated. The orchard trees are then normalized and treated as circular obstacles, and an orchard map is constructed. The hybrid path planning algorithm based on the improved D*Lite algorithm is then validated on this map.

To adapt to the soft terrain environment of orchards, a tracked differential steering chassis was selected, as shown in Figure 10. The chassis is equipped with dual 650 W servo motors, providing a maximum driving load of 220 kg, ensuring the robot's agile mobility.

This study uses a central computer running the Ubuntu 22.04 LTS operating system and is developed based on the Robot Operating System 2 (ROS2). The robot's position in the global coordinate system is first obtained through a combined navigation module. The relative positions of orchard trees to the robot are acquired using a 3D LiDAR sensor, and after coordinate transformation, the global coordinates of the trees are determined. The trees are then regularized and represented as circular obstacles to construct the orchard map. This map serves as the basis for validating the hybrid path planning algorithm based on the improved D* Lite algorithm.



Figure 10. Schematic diagram of the tracked chassis.

3.2.2. Experimental Design

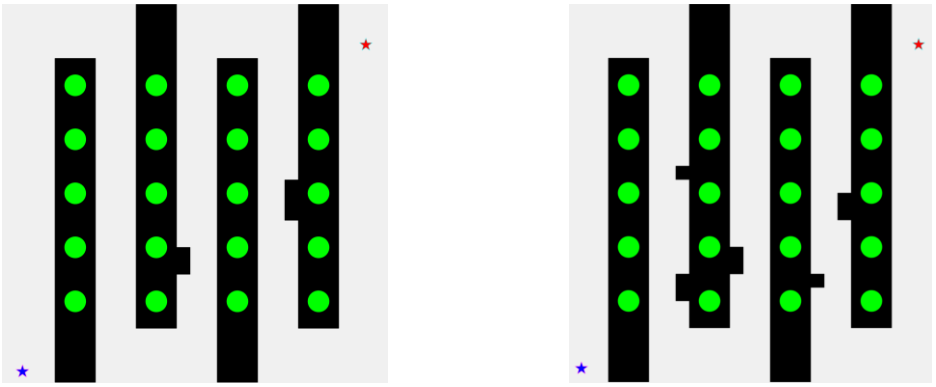
To verify the practicality and robustness of the proposed improved D* Lite-based hybrid path planning algorithm in a structured orchard environment, a field experiment was conducted on 27 June 2025, in a wooded area on the campus of Jiangsu University (geographical coordinates: 119°30′45.709″ E, 32°11′55.061″ N), as shown in Figure 11. The trees in this area are arranged in a regular pattern, with row spacing of 3.0 m and plant spacing of 2.2 m, effectively simulating a typical orchard driving environment.



(**a**) (**b**)

Figure 11. Experimental scenario: (**a**) the blue box indicates the selected experimental area, and the red box marks the obstacles in the scene, with humans simulating newly added obstacles. The blue circle represents the obstacles that were originally present; (**b**) On-site view of the experiment in progress.

Based on the test site, a structurally continuous corridor-style map was constructed, as shown in Figure 12.



(**a**) (**b**)

Figure 12. Navigation maps for physical experiments: (**a**) original forest navigation map; (**b**) navigation map after introducing new obstacles.

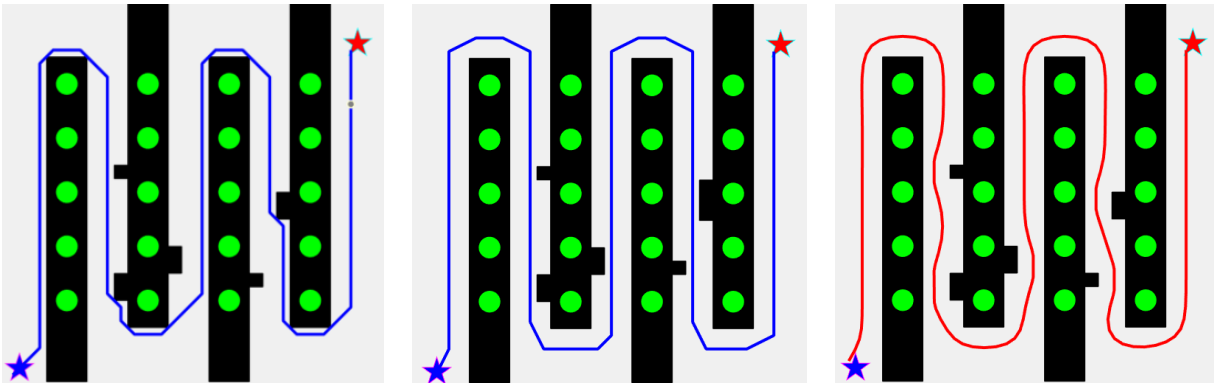
The traditional D* Lite algorithm, KD* Lite-B, and the proposed improved D* Lite-based hybrid algorithm were each run on the above map. The global path planning trajectories generated by these three methods were compared and visualized using Rviz, as shown in Figure 13.

Figure 13a presents the path generated by the traditional D* Lite algorithm. It can be observed that the path exhibits typical grid-based search characteristics, featuring noticeable sharp turns, especially at corridor corners or near obstacle edges, resulting in poor smoothness. Such discontinuous paths cause frequent direction changes during real-world tracking by the differential tracked robot, adversely affecting its motion stability. Additionally, the path often runs close to obstacle boundaries, posing collision risks and safety concerns.

Figure 13b shows the path planned by the KD* Lite-B algorithm. As illustrated, this path is smoother and more regular compared to the traditional method, with fewer sharp angles at turns and a trajectory closer to the center of the tree rows, thereby enhancing safety and executability. However, the algorithm has limited adaptability to dynamic environments, and in certain complex areas, it still produces paths that approach obstacles

too closely. In highly structured and spatially constrained orchard environments, KD* Lite-B leaves room for improvement in fine path optimization and dynamic response.

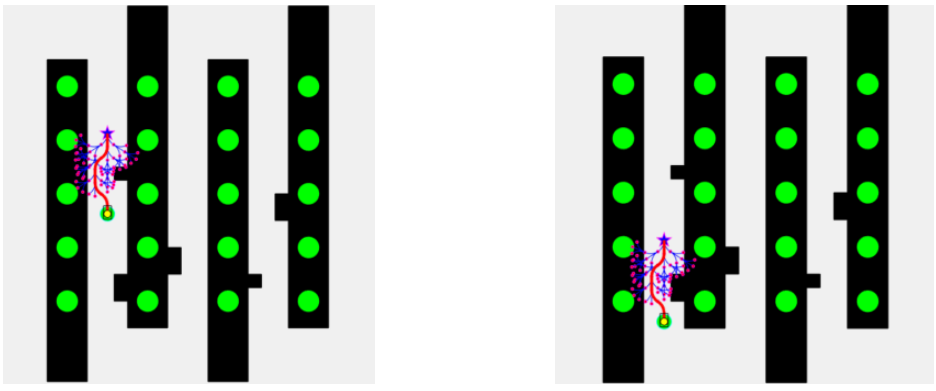
Figure 13c displays the planning results of the proposed hybrid algorithm based on the improved D* Lite. Building on the improved D* Lite path, this method further detects and identifies narrow corridor segments, applying the Hybrid A* algorithm locally within these narrow regions. The Hybrid A* not only incorporates non-holonomic constraints but also generates continuously differentiable curved trajectories. As shown, the hybrid algorithm demonstrates evident smooth curve transitions in key areas such as turns and narrow passages, significantly enhancing path smoothness and improving practical executability on the robot platform.



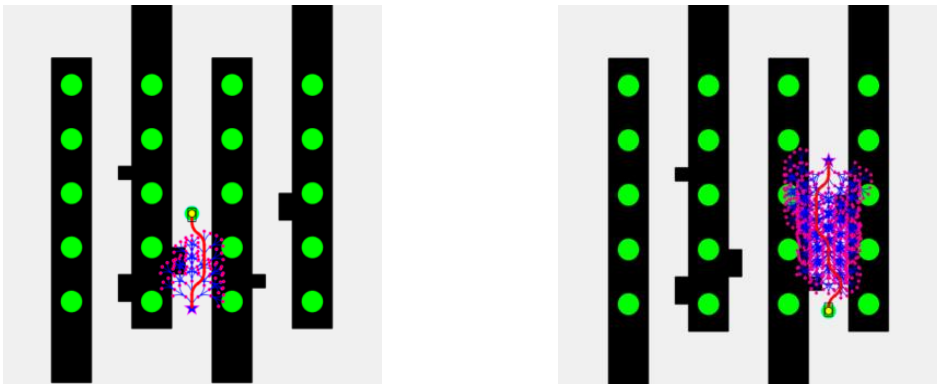
(**a**) (**b**) (**c**)

Figure 13. Path planning results in real-world scenario tests: (**a**) traditional D* Lite algorithm with blue lines; (**b**) KD* Lite-B algorithm with blue lines; (**c**) hybrid algorithm based on improved D* Lite with red lines.

To further demonstrate the hybrid algorithm's capability in handling narrow passages within orchard environments, Figure 14 presents the local path replanning results using the Hybrid A* method in spatially constrained corridor areas. The path shown is based on the initial global path generated by the improved D* Lite algorithm. Upon detecting regions where the corridor width is insufficient, the hybrid algorithm automatically invokes Hybrid A* for local trajectory replacement and optimization. As observed in the figure, the hybrid algorithm generates continuous, smooth, and non-holonomic-compliant curved paths suitable for differential tracked robots within the narrow regions. The curve transitions are natural and effectively avoid the sharp turns characteristic of traditional grid-based searches. The path reasonably avoids obstacles within the constrained space and smoothly connects with the global paths at both ends, demonstrating strong practical executability and safety.



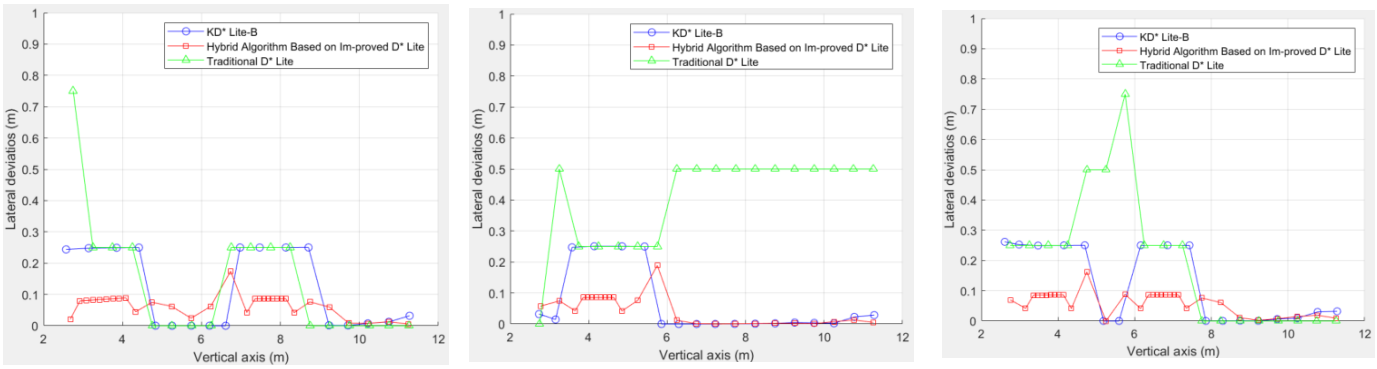
(**a**) (**b**)



(**c**) (**d**)

Figure 14. The actual scenario test results for path optimization in narrow passage areas: (a) re-planning result for narrow section 1; (b) re-planning result for narrow section 2; (c) re-planning result for narrow section 3; (d) re-planning result for narrow section 4. The blue curve represents the exploration path of the Hybrid A* algorithm during the search process, while the red line represents the optimized path, which is the re-planning result, the yellow circle marks the endpoint of the local re-planning.

Since the primary operating area of orchard robots lies between tree rows, this study focuses on comparing the lateral deviations of trajectories generated by three path planning algorithms with respect to the centerlines of the inter-row spaces, excluding the turning maneuvers between rows. Figure 15 illustrates the deviation performance of the traditional D* Lite, KD* Lite-B, and the proposed improved algorithm. As shown in the figure, the proposed algorithm consistently exhibits smaller lateral deviations across all inter-row sections. In particular, near narrow and constrained areas, its trajectory closely adheres to the centerline of the tree rows, demonstrating significantly better alignment and precision compared to the other two algorithms.



(**a**) (**b**) (**c**)

Figure 15. Comparison of lateral deviations from tree row centerlines in real-world test scenarios: (**a**) first inter-row space; (**b**) second inter-row space; (**c**) third inter-row space.

The global path deviations from the tree row centerlines for each algorithm are summarized in Table 4. As evident from the data, the proposed hybrid algorithm based on improved D* Lite significantly reduces lateral deviations across all inter-row sections. In the first inter-row, the hybrid algorithm achieves a deviation of 0.0670 m, representing a reduction of approximately 51.8% compared to the 0.1389 m of the traditional D* Lite, and is on par with KD* Lite-B. In the second inter-row, the deviation is reduced from 0.4028 m (traditional D* Lite) to 0.0487 m, yielding an 87.9% decrease and outperforming KD* Lite-B (0.0617 m) by approximately 21.1%. For the third inter-row, the deviation drops

from 0.1944 m to 0.0643 m, marking a 66.9% reduction, and is also about 44.7% lower than that of KD* Lite-B (0.1162 m).

Table 4. Comparison of lateral deviations from tree row centerlines in real-world test scenarios.

Algorithm	First Inter-Row Space(m)	Second Inter-Row Space(m)	Third Inter-Row Space(m)
Traditional D* Lite	0.1386	0.4028	0.1944
KD* Lite-B	0.1136	0.0617	0.1162
Hybrid algorithm based on improved D* Lite	0.0670	0.0487	0.0643

To comprehensively evaluate the adaptability and path quality of different path planning algorithms in structured orchard environments, this study compares traditional D* Lite, KD* Lite-B, and the proposed improved D* Lite-based hybrid algorithm based on four key performance indicators: average search time, number of risky nodes, path length, and overall average deviation from the centerline. The results are summarized in Table 5.

The results indicate that the improved hybrid algorithm outperforms the other methods in terms of path accuracy, safety, and execution stability. Although the average search time of the hybrid algorithm is 14.674 s, which is higher than KD* Lite-B with 8.153 s and traditional D* Lite with 10.138 s, the path quality is significantly enhanced. The number of risky nodes generated by the hybrid algorithm is only 5, representing a reduction of 68.75 percent compared to KD* Lite-B with 16 nodes and a decrease of 92.86 percent relative to the 70 nodes of traditional D* Lite, greatly improving path safety. Regarding path centrality, the hybrid algorithm achieves an average deviation from the centerline of 0.0600 m, approximately 38.3 percent less than KD* Lite-B with 0.0972 m and 75.5 percent less than traditional D* Lite with 0.2454 m, effectively meeting the stringent precision requirements of orchard operations. In terms of path length, the hybrid algorithm produces a path length of 65.65 m, slightly shorter than KD* Lite-B with 66.56 m and marginally longer than traditional D* Lite with 61.52 m; however, due to its smoother and curvature-continuous trajectory, it demonstrates superior executability and practical applicability.

Table 5. Performance comparison of path planning algorithms in real-world experiments.

Algorithm	Average Search Time (s)	Number of Dangerous Nodes (Count)	Path Length (m)	Overall Mean Lateral Deviation from Centerline (m)
Traditional D* Lite	10.138	70	61.52	0.2454
KD* Lite-B	8.153	16	65.56	0.0972
Hybrid algorithm based on improved D* Lite	14.674	5	65.65	0.0600

4. Discussion

In this paper, the hybrid path planning algorithm based on the improved D* Lite*D Lite** algorithm demonstrates significant advantages in path safety, path centrality, and local path optimization. Compared to traditional D* Lite and KD* Lite-B algorithms, our hybrid algorithm achieves notable improvements in reducing path deviation and the number of hazardous nodes.

Firstly, path centrality is a key advantage of our algorithm. The traditional D* Lite algorithm, due to the lack of structured path guidance, results in a more noticeable deviation from the centerline of the orchard rows. In contrast, the hybrid algorithm introduces Voronoi guidance and a centerline deviation penalty term, making the path more closely align with the centerline of the orchard rows. This greatly improves the structural rationality of the path and enhances its adaptability in complex environments.

Secondly, in safety assessment, the hybrid algorithm significantly reduces the number of hazardous nodes in the path through local replanning. This demonstrates that our algorithm is more robust in handling high-risk areas, effectively avoiding collision risks, and improving the safety of the path.

Although the computational time of the hybrid algorithm is slightly longer, considering the significant improvements in path quality, safety, and execution stability, this increase in computational cost is acceptable. Particularly in practical operations, the reduced frequency of path corrections and lower control costs can significantly improve the system's efficiency and robustness.

The hybrid algorithm proposed in this paper excels in several aspects, especially in terms of structural adaptability and path safety. This provides new directions for future research, particularly in more complex dynamic environments, where the advantages of the algorithm will be even more prominent.

5. Limitations

Although the hybrid path planning algorithm proposed in this study demonstrates superior performance in several aspects, there are still some limitations. First, the current experimental environment is primarily focused on orchard scenarios. While the algorithm performs well in this environment, its adaptability in more complex dynamic environments still needs further validation. The environmental changes used in the orchard experiments are typically local and temporary, such as fallen branches or changes in the placement of items in the orchard, and do not involve long-term or continuous dynamic obstacles. This means that the algorithm's performance and adaptability in a continuously changing dynamic environment have not been fully assessed. Therefore, future research should extend the validation to more challenging environments. Secondly, although local path optimization and curve processing steps have been introduced, the hybrid algorithm incurs a higher computational cost compared to traditional algorithms. Future efforts can focus on further optimizing algorithm efficiency and reducing computation time. Additionally, considering the presence of different types of obstacles and variable terrain in the environment, the generalizability and adaptability of the algorithm still require further testing and validation.

6. Cost-Effectiveness and Practical Implications

From a cost-effectiveness perspective, although the hybrid algorithm results in increased search time, the improvements in path quality, safety, and execution stability have a positive impact on practical operations. By reducing the frequency of path corrections and lowering control costs, the hybrid algorithm effectively enhances the overall efficiency and robustness of the orchard robot system. In particular, the safety of the path is

significantly improved in high-risk areas, helping to reduce collision risks during operations and enhancing the system's long-term stability and operational accuracy. Furthermore, the implementation of this algorithm reduces manual intervention and path corrections, increasing the level of automation in operations, thereby lowering operational costs.

7. Conclusions

This study proposes a hybrid path planning algorithm based on an improved D* Lite method, aiming to enhance the autonomous navigation performance of orchard robots in structured orchard environments. The method integrates Voronoi diagram structural information to extract the centerlines of orchard rows and designs a composite heuristic cost function incorporating centerline deviation penalties, turning smoothness terms, and obstacle gradient awareness, thereby significantly improving path search efficiency and structural adaptability. By detecting narrow passages along the initial path, the algorithm introduces a hybrid A* local optimization module that considers the kinematic constraints of differential tracked robots, achieving coordinated enhancement of path continuity and executability.

Field experiments demonstrate that the proposed hybrid algorithm outperforms the traditional D* Lite and KD* Lite-B algorithms in key metrics, including path accuracy, safety, and operational stability. Specifically, the average lateral deviation from the centerline is 0.0600 m, representing reductions of 75.55% and 38.27% compared to the traditional D* Lite (0.2454 m) and KD* Lite-B (0.0972 m), respectively. The number of risky nodes decreases to 5, marking decreases of 92.86% and 68.75% relative to D* Lite (70 nodes) and KD* Lite-B (16 nodes), respectively. Although the search time increases slightly, the algorithm achieves improved path smoothness and trajectory executability, better meeting the practical requirements of orchard robots.

The algorithm has been successfully applied to an autonomous tracked spraying robot platform. Experimental results confirm its high robustness and environmental adaptability when dealing with unexpected obstacles, spatial constraints, and path feasibility issues in orchard environments. Future research will focus on enhancing the algorithm's online dynamic responsiveness, introducing multi-robot coordination mechanisms, and integrating high-precision perception with teleoperation systems to develop a more complete intelligent orchard operation framework. In conclusion, the proposed hybrid path planning system offers a scalable and practical solution for achieving efficient, reliable, and autonomous navigation in agricultural robots, with broad potential for real-world applications.

Author Contributions: Conceptualization, Q.J. and Y.S.; methodology, Q.J.; Software, Q.J.; Resources, H.L. and Y.S.; Supervision, H.L. and Y.S.; validation, Q.J., Y.S., and H.L.; formal analysis, Q.J., H.S., and Y.H.; investigation, Q.J., Z.K., and Y.H.; data curation, Q.J. and Y.S.; writing—original draft preparation, Q.J.; writing—review and editing, Q.J., Z.K., and H.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the National Natural Science Foundation of China, grant number 32171908, and the Jiangsu Provincial Agricultural Science and Technology Independent Innovation Fund, grant number CX(24)3025.

Data Availability Statement: The data presented in this study are available on request from the corresponding author. The data are not publicly available due to institutional restrictions and ongoing intellectual property considerations related to their use in agricultural automation applications.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Ma, Z.; Yang, S.; Li, J.; Qi, J. Research on slam localization algorithm for orchard dynamic vision based on YOLOD-SLAM2. *Agriculture* **2024**, *14*, 1622.
2. Jiangyi, H.; Fan, W. Design and testing of a small orchard tractor driven by a power battery. *Eng. Agrícola* **2023**, *43*, e20220195.
3. Ji, W.; Gao, X.; Xu, B.; Pan, Y.; Zhang, Z.; Zhao, D. Apple target recognition method in complex environment based on improved YOLOv4. *J. Food Process Eng.* **2021**, *44*, e13866.
4. Khan, Z.; Liu, H.; Shen, Y.; Yang, Z.; Zhang, L.; Yang, F. Optimizing precision agriculture: A real-time detection approach for grape vineyard unhealthy leaves using deep learning improved YOLOv7 with feature extraction capabilities. *Comput. Electron. Agric.* **2025**, *231*, 109969.
5. Ji, W.; Zhang, T.; Xu, B.; He, G. Apple recognition and picking sequence planning for harvesting robot in a complex environment. *J. Agric. Eng.* **2024**, *55*, 1549.
6. Li, J.; Shang, Z.; Li, R.; Cui, B. Adaptive sliding mode path tracking control of unmanned rice transplanter. *Agriculture* **2022**, *12*, 1225.
7. Cui, B.; Cui, X.; Wei, X.; Zhu, Y.; Ma, Z.; Zhao, Y.; Liu, Y. Design and Testing of a Tractor Automatic Navigation System Based on Dynamic Path Search and a Fuzzy Stanley Model. *Agriculture* **2024**, *14*, 2136.
8. Khan, Z.; Shen, Y.; Liu, H. Object Detection in Agriculture: A Comprehensive Review of Methods, Applications, Challenges, and Future Directions. *Agriculture* **2025**, *15*, 1351.
9. Khan, Z.; Liu, H.; Shen, Y.; Zeng, X. Deep learning improved YOLOv8 algorithm: Real-time precise instance segmentation of crown region orchard canopies in natural environment. *Comput. Electron. Agric.* **2024**, *224*, 109168.
10. Jin, Y.; Liu, J.; Xu, Z.;Yuan, S.; Li, P.; Wang, J. Development status and trend of agricultural robot technology. *Int. J. Agric. Biol. Eng.* **2021**, *14*, 1–19.
11. Jia, W.; Tai, K.; Wang, X.; Dong, X.; Ou, M. Design and simulation of intra-row obstacle avoidance shovel-type weeding machine in orchard. *Agriculture* **2024**, *14*, 1124.
12. Liu, W.; Zhou, J.; Liu, Y.; Zhang, T.; Yan, M.; Chen, J.; Zhou, C.; Hu, J.; Chen, X. An Ultrasonic Ridge-Tracking Method Based on Limiter Sliding Window Filter and Fuzzy Pure Pursuit Control for Ridge Transplanter. *Agriculture* **2024**, *14*, 1713.
13. Lu, E.; Xue, J.; Chen, T.; Jiang, S. Robust trajectory tracking control of an autonomous tractor-trailer considering model parameter uncertainties and disturbances. *Agriculture* **2023**, *13*, 869.
14. Sun, J.; Wang, Z.; Ding, S.; Jiang, S. Adaptive disturbance observer-based fixed time nonsingular terminal sliding mode control for path-tracking of unmanned agricultural tractors. *Biosyst. Eng.* **2024**, *246*, 96–109.
15. Zhu, S.; Wang, B.; Pan, S.; Xia, J.; Xing, G. Task allocation of multi-machine collaborative operation for agricultural machinery based on the improved fireworks algorithm. *Agronomy* **2024**, *14*, 710.
16. Xie, F.; Guo, Z.; Li, T.; Feng, Q.; Zhao, C. Dynamic Task Planning for Multi-Arm Harvesting Robots Under Multiple Constraints Using Deep Reinforcement Learning. *Horticulturae* **2025**, *11*, 88.
17. Chen, T.; Xu, L.; Ahn, H.S.; Lu, E.; Liu, Y.; Xu, R. Evaluation of headland turning types of adjacent parallel paths for combine harvesters. *Biosyst. Eng.* **2023**, *233*, 93–113.
18. Shi, R.; Han, X.; Guo, W. Uncertain multi-objective programming approach for planning supplementary irrigation areas in rainfed agricultural regions. *Irrig. Drain.* **2024**, *74*, 1193–1214.
19. Lu, E.; Xu, L.; Li, Y.; Tang, Z.; Ma, Z.Modeling of working environment and coverage path planning method of combine harvesters. *Int. J. Agric. Biol. Eng.* **2020**, *13*, 132–137.
20. Zhong, W.; Yang, W.; Zhu, J.; Jia, W.; Dong, X.; Ou, M. An Improved UNet-Based Path Recognition Method in Low-Light Environments. *Agriculture* **2024**, *14*, 1987.
21. Liu, H.; Zeng, X.; Shen, Y.; Xu, J.; Khan, Z. A Single-Stage Navigation Path Extraction Network for agricultural robots in orchards. *Comput. Electron. Agric.* **2025**, *229*, 109687.
22. Zhang, F.; Teng, S.; Wang, Y.; Xu, J.; Khan, Z. Design of bionic goat quadruped robot mechanism and walking gait planning. *Int. J. Agric. Biol. Eng.* **2020**, *13*, 32–39.
23. Liu, W.; Hu, J.; Liu, J.; Yue, R.; Zhang, T.; Yao, M.; Li, J. Method for the navigation line recognition of the ridge without crops via machine vision. *Int. J. Agric. Biol. Eng.* **2024**, *17*, 230–239.
24. Zhang, T.; Zhou, J.; Liu, W.; Yue, R.; Shi, J.; Zhou, C.; Hu, J. SN-CNN: A Lightweight and Accurate Line Extraction Algorithm for Seedling Navigation in Ridge-Planted Vegetables. *Agriculture* **2024**, *14*, 1446.
25. Luo, Y.; Wei, L.; Xu, L.; Zhang, Q.; Liu, J.; Cai, Q.; Zhang, W. Stereo-vision-based multi-crop harvesting edge detection for precise automatic steering of combine harvester. *Biosyst. Eng.* **2022**, *215*, 115–128.

26. Sang, Y.; Chen, X.; Chen, Q.; Tao, J.; Fan, Y. A route planning for oil sample transportation based on improved A* algorithm. *Sci. Rep.* **2023**, *13*, 22041.
27. Wu, Z.; Meng, Z.; Zhao, W.; Wu, Z. Fast-RRT: A RRT-based optimal path finding method. *Appl. Sci.* **2021**, *11*, 11777.
28. Liu, X.; Deng, R.; Wang, J.; Wang, X. COStar: A D-star Lite-based dynamic search algorithm for codon optimization. *J. Theor. Biol.* **2014**, *344*, 19–30.
29. Luo, Z.; Zhang, Y.; Mu, L.; Huang, J.; Xin, J.; Liu, H.; Jiao, S.; Xie, G.; Yi, Y. A UAV Path Planning Algorithm Based on an Improved D* Lite Algorithm for Forest Firefighting. In Proceedings of the Chinese Automation Congress (CAC), Shanghai, China, 6–8 November 2020; pp. 4233–4237.
30. Jin, J.; Zhang, Y.; Zhou, Z.; Jin, M.; Yang, X.; Hu, F. Conflict-based search with D* lite algorithm for robot path planning in unknown dynamic environments. *Comput. Electr. Eng.* **2023**, *105*, 108473.
31. Gao, Y.; Han, Q.; Feng, S.; Wang, Z.; Meng, T.; Yang, J. Improvement and fusion of D* lite algorithm and dynamic window approach for path planning in complex environments. *Machines* **2024**, *12*, 525.
32. Li, X.; Lu, Y.; Zhao, X.; Deng, X.; Xie, Z. Path planning for intelligent vehicles based on improved D* Lite. *J. Supercomput.* **2024**, *80*, 1294–1330.
33. Koenig, S.; Likhachev, M. D* lite. In Proceedings of the Eighteenth National Conference on Artificial Intelligence, Edmonton, AB, Canada, 28 July–1 August 2002; pp. 476–483.
34. Dolgov, D.; Thrun, S.; Montemerlo, M.; Diebel, J. Practical search techniques in path planning for autonomous driving. *Ann Arbor* **2008**, *1001*, 18–80.
35. Aurenhammer, F. Voronoi diagrams—A survey of a fundamental geometric data structure. *ACM Comput. Surv.* **1991**, *23*, 345–405.
36. Sani, E. ROS 2-Based Autonomous Navigation for Agricultural Robots: Sensor Integration and Processing for Traversable Terrain Segmentation. Master's Thesis, University of Genova, Genoa, Italy, 2023.
37. Saipullah, K.M.; Saad, W.H.M.; Ramlee, F.N.I.; Idris, M.I.; Din, M.A.F.M. Development of delta robot arm simulation in ROS2 Foxy Fitzroy environment. *J. Telecommun. Electron. Comput. Eng.* **2022**, *14*, 1–6.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.